AI-Assisted Software Development

Module 4: Refinement and Quality with AI Tools

Mike Burton

Overview

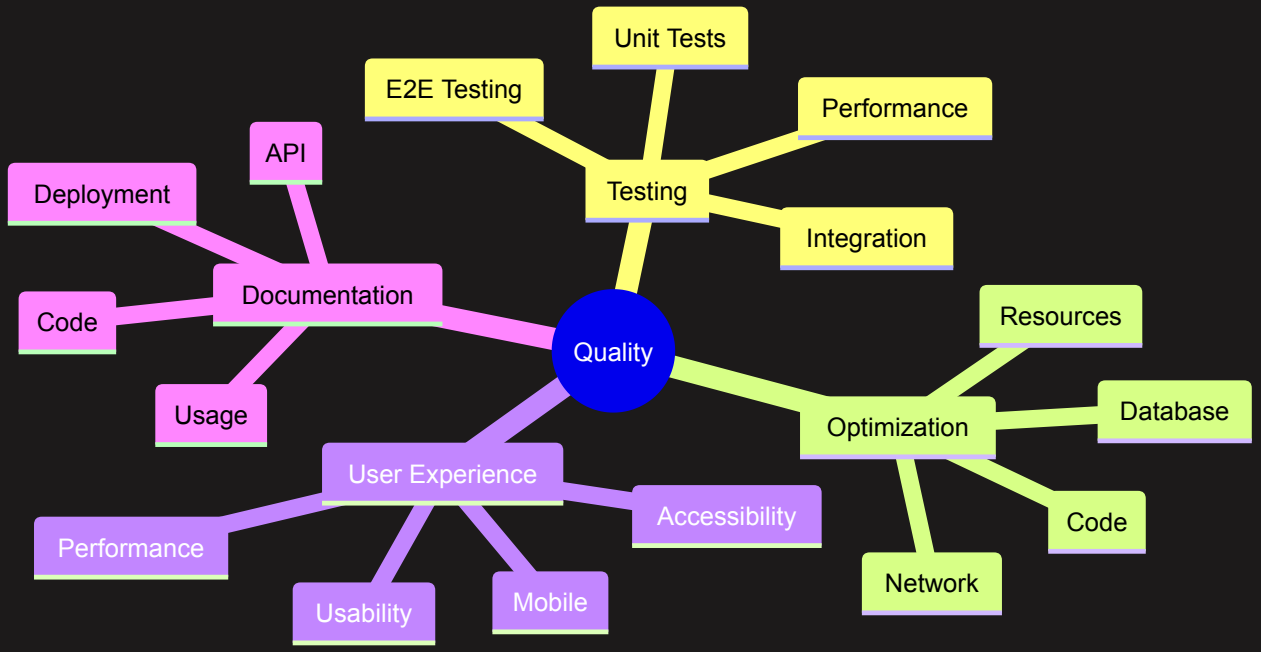
What we'll cover:

- Practical testing approaches
- Real-world optimization
- Code quality improvements
- UX refinement
- Production readiness

Using AI tools for:

- Test generation
- Performance analysis
- Code review
- UX validation
- Security checks

The Quality Process



AI-Assisted Test Generation

Example prompt
for test case
generation:

```
Generate unit tests for the authentication module.
(Include context if needed.)
```

You can also add specifications and constraints. Examples:

```
- Ensure the output is in JSON format,
  similar to this example: ...
- Create tests for edge cases, such
  as empty input, invalid email format, etc.
- Create end-to-end tests for the critical
  path, such as a user logging in and out.
- Create data-driven tests for
  this function: ...
```

1. Testing Strategy

```
Ask AI to review your testing approach:

"Review this testing strategy for gaps:
- Unit tests for utilities
- Integration tests for API
- E2E tests for critical paths
- Performance tests for API endpoints"
```

Performance Analysis Prompts

Real-world examples:

1. Query Optimisation

```
Analyze this query for performance issues:

SELECT u.*, p.*, c.*
FROM users u
LEFT JOIN posts p ON u.id = p.user_id
LEFT JOIN comments c ON p.id = c.post_id
WHERE u.status = 'active'
ORDER BY p.created_at DESC;
```

2. Frontend Performance

```
- Review this React component for performance: ...
```

Code Quality Prompts

Example prompts
for code review:

1. Pattern Review

```
Review this code for potential patterns
and suggest improvements: ...
```

2. Code Review

```
Use AI for code review:

"Review this PR for:
- Performance issues
- Security concerns
- Best practices
- Potential bugs
- Edge cases
- Error handling"
```

32. Error Handling

```
Improve error handling in this code:

async function processOrder(orderId) {
  const order = await getOrder(orderId);
  const payment = await processPayment(order);
  await updateInventory(order);
  await sendConfirmation(order);
  return order;
}
```

UX Refinement Examples

Real-world UX improvement prompts:

1. Form Validation

```
Suggest improvements for this form:

<form onSubmit={handleSubmit}>
  // ...
  <button type="submit">Submit</button>
</form>
```

2. Error Messages

```
Review these error messages for user-friendliness:

- "Error 404: Resource not found"
- "Database connection failed"
- "Invalid input in field 'email'"
```

Security Review Prompts

Example security analysis:

1. Authentication Flow

```
Review this authentication flow for security issues:

1. User submits email/password
2. Backend validates credentials
3. Generate JWT token
4. Store token in localStorage
5. Include token in Authorization header
```

2. Data Validation

```
Check this input validation for security:

function processUserInput(data) {
  // ...
}
```

Performance Optimisation Examples

Real-world optimization prompts:

1. React Component

```
Optimize this component for performance:

function ProductList({ products }) {
  // ...
}
```

2. API Endpoint

```
Optimize this API endpoint for performance:

app.get('/api/users', async (req, res) => {
  // ...
});
```

Error Handling Improvements

Example error handling prompts:

1. API Error Handling

```
Improve error handling in this API:

app.post('/api/orders', async (req, res) => {
  try {
    // ...
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

2. Frontend Error Boundaries

```
Review this error boundary implementation:

class ErrorBoundary extends React.Component {
  // ...
}
```

Documentation Examples

Real-world documentation prompts:

1. API Documentation

```
Generate comprehensive documentation
for this endpoint:

POST /api/users
Body: {
  name: string
  email: string
  role?: 'user' | 'admin'
  preferences?: {
    theme: 'light' | 'dark'
    notifications: boolean
  }
}
```

2. Component Documentation

```
Document this React component:

function DataTable({
  data,
  columns,
  sortable = true,
  filterable = false,
  pagination = true,
  onRowClick,
  onSort,
  onFilter
}) {
  // Component implementation
}
```

Common Pitfalls and Solutions

1. Performance Issues

```
Ask AI to identify problems:

"Review this code for performance issues:
- Unnecessary re-renders
- N+1 queries
- Memory leaks
- Unoptimized assets"
```

2. Security Concerns

```
Use AI for security review:

"Check this implementation for:
- SQL injection risks
- XSS vulnerabilities
- CSRF protection
- Authentication bypass"
```

Recap

- Use specific, detailed prompts
- Include context and requirements
- Ask for multiple approaches
- Validate AI suggestions
- Iterate on improvements
- Document decisions
- Test thoroughly

Next Steps

- Apply these prompts to your code
- Create your prompt library
- Build testing templates
- Establish review checklist
- Set up monitoring